

Lesson 16 Structured Note: Program for Infix-to-Postfix Conversion

0. Current Position in the Course

You are now in **Lesson 16** of the stack curriculum.

Before this lesson, you already learned several important ideas:

Lesson	Main idea
Lesson 1	A stack follows LIFO: Last In, First Out .
Lesson 2	An array stack uses <code>size</code> , <code>top</code> , and an array pointer.
Lesson 3	Array stack operations include <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .
Lesson 4	A complete C array-stack program combines all array-stack functions.
Lesson 5	A linked-list stack uses nodes and a <code>top</code> pointer.
Lesson 6	Linked-list stack operations include <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .
Lesson 7	A complete C linked-list stack uses <code>struct Node</code> , a global <code>top</code> pointer, <code>push</code> , <code>pop</code> , <code>display</code> , and <code>main</code> .
Lesson 8	A C++ class can be used to implement a linked-list stack more safely.
Lesson 9	Parenthesis matching was introduced as a stack application.
Lesson 10	Parenthesis matching was implemented using an array character stack.
Lesson 11	Parenthesis matching was implemented using a linked-list character stack.
Lesson 12	Bracket matching was extended to <code>()</code> , <code>[]</code> , and <code>{}</code> .
Lesson 13	You learned infix, prefix, postfix, precedence, and simple manual expression conversion.
Lesson 14	You learned associativity and unary operators.
Lesson 15	You learned the stack logic for converting infix to postfix.

Lesson 16 continues:

Phase 4: Expression Notation and Conversion

Lesson 15 focused on the **idea** of infix-to-postfix conversion.

Lesson 16 focuses on the **program**.

In simple words:

Lesson 15 taught what to do.

Lesson 16 teaches how to write the code carefully.

1. Lesson 16 Goal

By the end of this lesson, you should understand:

- What infix-to-postfix conversion means.
 - Why postfix is useful.
 - Why a stack is used during conversion.
 - How to build a program for infix-to-postfix conversion.
 - Why the program uses a character stack.
 - Why the postfix result is stored in a character array.
 - What helper functions are needed.
 - How `isOperand()` works.
 - How `precedence()` works.
 - Why we need two index variables, `i` and `j`.
 - Why `i` scans the infix expression.
 - Why `j` writes into the postfix expression.
 - Why operands go directly to postfix.
 - Why operators are pushed or popped using precedence.
 - Why a `while` loop is better than a `for` loop in this program.
 - Why `i` does not always increase immediately.
 - Why remaining operators are popped after the scan finishes.
 - Why the postfix expression must end with `\0`.
 - Why dynamically allocated arrays must be cleaned up.
 - How to dry-run the program.
 - What this beginner version can handle.
 - What this beginner version cannot handle yet.
 - The time and space complexity of the program.
-

2. Quick Review: What Is an Expression?

An **expression** is a combination of operands and operators.

Example:

A + B

This expression has:

Symbol	Role
A	Operand
+	Operator
B	Operand

3. Operand

An **operand** is a value or variable used in an expression.

Examples of operands:

A, B, C, x, y, a, b, c, 5, 9

In this expression:

a + b

The operands are:

a and b

4. Operator

An **operator** is a symbol that performs an operation.

Common operators are:

Operator	Meaning
$+$	Addition
$-$	Subtraction
$*$	Multiplication
$/$	Division

In this expression:

$a + b$

The operator is:

$+$

5. Quick Review: Infix Notation

In **infix notation**, the operator is written between the operands.

Structure:

Operand Operator Operand

Example:

$a + b$

Here, $+$ is between a and b .

So this is infix notation.

Humans usually write expressions in infix form.

Examples:

$a + b$
 $a - b$

a * b
a / b
a + b * c

6. Quick Review: Postfix Notation

In **postfix notation**, the operator is written after the operands.

Structure:

Operand Operand Operator

Example:

a b +

This means the same thing as:

a + b

The operator **+** comes after **a** and **b**.

So this is postfix notation.

Examples:

Infix	Postfix
a + b	ab+
a - b	ab-
a * b	ab*
a / b	ab/

Important idea:

Infix and postfix can represent the same expression.
Only the position of the operator changes.

7. What Problem Are We Solving?

Humans usually write expressions like this:

$$a + b * c$$

This is infix notation.

But computers can evaluate postfix expressions more easily.

The postfix form of:

$$a + b * c$$

is:

$$abc^{*+}$$

Why?

Because multiplication has higher precedence than addition.

So:

$$a + b * c$$

really means:

$$a + (b * c)$$

In postfix:

$$b * c$$

becomes:

$$bc^{*}$$

Then we add `a`:

```
abc*+
```

So Lesson 16 asks this question:

How do we write a program that converts an infix expression into postfix form?

8. Why Do We Need a Stack?

In an infix expression, operators do not always execute in the order we see them.

Example:

```
a + b * c
```

When scanning left to right, we see `+` before `*`.

But `*` must be done first because multiplication has higher precedence.

So the `+` operator must wait.

This is where the stack helps.

The stack temporarily stores operators until it is safe to place them in the postfix result.

Simple idea:

```
Operands go directly to postfix.  
Operators wait in the stack.
```

The stack is not the final answer.

The stack is only a temporary waiting area for operators.

9. Main Rule from Lesson 15

Lesson 15 gave this main rule:

```
Operand -> send directly to postfix  
Operator -> compare with stack top  
End      -> pop remaining operators
```

Lesson 16 turns this rule into code.

10. Big Idea of Lesson 16

Lesson 16 is about writing a real program for this conversion.

Instead of only thinking:

```
a+b*c  ->  abc*+
```

we write a function that receives:

```
char infix[] = "a+b*c";
```

and returns:

```
"abc*+"
```

So Lesson 16 is more code-focused than Lesson 15.

11. What the Program Needs

The program needs four main parts:

1. A stack for operators
2. A postfix character array for the answer
3. Helper functions
4. Two index variables: i and j

The operator stack temporarily stores operators such as:

```
+ - * /
```


The postfix array stores the final converted expression:

```
abc*+
```

12. Why the Stack Stores Characters

Earlier stack examples stored integers, such as:

```
10, 20, 30
```

But in infix-to-postfix conversion, we are storing symbols from an expression.

These symbols are characters:

```
'a', 'b', '+', '*'
```

So our stack stores `char` values.

The array inside the stack is:

```
char *S;
```

not:

```
int *S;
```

Important idea:

The stack rule does not change.
Only the data type changes.

13. Stack Structure Used in Lesson 16

We can define the stack like this:

```
struct Stack {  
    int size;
```

```
int top;  
char *S;  
};
```

Meaning of each field:

Field	Simple meaning
size	Maximum number of characters the stack can store.
top	Index of the current top character.
S	Pointer to the character array used as the stack.

Example:

```
Stack  
├ size = 5  
├ top = -1  
└ S -> character array
```

14. Empty Condition

Because this is still an array stack, the empty condition is:

```
top == -1
```

This means the stack has no active element.

Example:

```
top = -1  
stack = empty
```

Important warning:

```
top = -1 does not mean S[-1] exists.
```

Array index `-1` is invalid.

It is only a marker meaning:

The stack is empty.

15. Full Condition

The stack is full when:

```
top == size - 1
```

Example:

```
size = 5  
valid indices = 0, 1, 2, 3, 4
```

The last valid index is:

4

So the stack is full when:

```
top == 4
```

16. Helper Function: isEmpty()

The `isEmpty()` function checks whether the stack has no active elements.

Code:

```
int isEmpty(Stack st) {  
    return st.top == -1;  
}
```

Return values:

Return value	Meaning
1	Stack is empty.

Return value	Meaning
0	Stack is not empty.

Example:

```
top = -1
```

Then:

```
isEmpty(st)
```

returns:

```
1
```

because the stack is empty.

17. Helper Function: `isFull()`

The `isFull()` function checks whether the stack has reached its maximum capacity.

Code:

```
int isFull(Stack st) {
    return st.top == st.size - 1;
}
```

Return values:

Return value	Meaning
1	Stack is full.
0	Stack is not full.

Example:

```
size = 5  
top = 4
```

Check:

```
top == size - 1  
4 == 5 - 1  
4 == 4
```

This is true.

So the stack is full.

18. Push Operation

The `push()` function inserts an operator into the stack.

Code:

```
void push(Stack *st, char x) {  
    if (isFull(*st)) {  
        cout << "Stack Overflow\n";  
    } else {  
        st->top++;  
        st->S[st->top] = x;  
    }  
}
```

A successful push does two things:

1. Increase top by 1.
2. Store the character at S[top].

Example:

Before pushing `+`:

```
top = -1  
stack = empty
```

After:

```
push(&st, '+');
```

we get:

```
top = 0  
S[0] = '+'
```

19. Why `push()` Uses `Stack *st`

The `push()` function changes the real stack.

It changes:

```
st->top  
st->S[st->top]
```

So it must receive the address of the stack:

```
Stack *st
```

That is why we use the arrow operator:

```
st->top  
st->S
```

20. Pop Operation

The `pop()` function removes and returns the top character from the stack.

Code:

```
char pop(Stack *st) {  
    char x = '\0';
```

```

    if (isEmpty(*st)) {
        cout << "Stack Underflow\n";
    } else {
        x = st->S[st->top];
        st->top--;
    }

    return x;
}

```

A successful pop does three things:

1. Save S[top].
2. Decrease top by 1.
3. Return the saved character.

Example:

Before pop:

```

S[0] = '+'
top = 0

```

After:

```

pop(&st);

```

we get:

```

returned character = '+'
top = -1
stack = empty

```

21. Why pop() Uses Stack *st

The pop() function changes the stack because it decreases top.

So it must receive the stack by pointer:

```
Stack *st
```

If we passed the stack by value, only a copy would change.

The real stack would not be updated.

22. Helper Function: `stackTop()`

The `stackTop()` function reads the current top operator without removing it.

Code:

```
char stackTop(Stack st) {  
    if (isEmpty(st))  
        return '\0';  
  
    return st.S[st.top];  
}
```

If the stack is empty, it returns:

```
'\0'
```

If the stack is not empty, it returns:

```
st.S[st.top]
```

Example:

```
stack = + *  
top element = *
```

Then:

```
stackTop(st)
```

returns:

*

Important:

`stackTop()` only reads.
It does not remove anything.

23. Helper Function: `isOperand()`

The `isOperand()` function checks whether a character is an operand.

For this beginner lesson:

a, b, c, x, y, 5, 9 -> operands
+, -, *, / -> operators

A simple version is:

```
int isOperand(char x) {  
    if (x == '+' || x == '-' || x == '*' || x == '/')  
        return 0;  
    return 1;  
}
```

Meaning:

Return value	Meaning
1	It is an operand.
0	It is an operator.

Examples:

```
isOperand('a') returns 1  
isOperand('b') returns 1  
isOperand('+') returns 0  
isOperand('*') returns 0
```

Simple idea:

If it is not one of + - * /, treat it as an operand.

24. Important Limitation of This `isOperand()` Function

This beginner version is simple.

It treats anything that is not `+`, `-`, `*`, or `/` as an operand.

So it works for simple expressions like:

```
a+b*c
```

But it does not properly handle:

```
spaces
parentheses
multi-digit numbers
unary minus
power operator
```

For example, in this beginner version, a space character could be treated like an operand.

A more advanced version would check letters and digits more carefully.

But for Lesson 16, the goal is to understand the basic conversion program.

25. Helper Function: `precedence()`

The `precedence()` function gives priority numbers to operators.

Code:

```
int precedence(char x) {
    if (x == '+' || x == '-')
        return 1;
    else if (x == '*' || x == '/')
        return 2;
```

```
    return 0;  
}
```

Meaning:

Operator	Precedence
+	1
-	1
*	2
/	2
Other character	0

So:

```
* is stronger than +  
/ is stronger than -  
+ and - are equal  
* and / are equal
```

26. Why `precedence()` Returns 0 for Other Characters

The function returns `0` for anything that is not an operator handled in this lesson.

Example:

```
precedence('\0') returns 0
```

This is useful because if the stack is empty, `stackTop()` may return `\0`.

Then:

```
precedence(stackTop(st))
```

becomes:

```
precedence('\0')
```

which returns `0`.

So if the current operator is `+`, then:

```
precedence('+') > precedence('\0')
1 > 0
```

This is true.

So the operator can be pushed.

27. The Two Index Variables: `i` and `j`

Lesson 16 uses two important index variables:

```
int i = 0;
int j = 0;
```

They do different jobs.

Variable	Job
<code>i</code>	Scans the infix expression.
<code>j</code>	Writes into the postfix expression.

Simple meaning:

```
i = reading finger
j = writing finger
```

28. Meaning of `i`

The variable `i` points to the current character in the infix expression.

Example:

```
infix: a + b * c
      ^
      i
```

At the beginning:

```
i = 0
infix[i] = 'a'
```

When we move to the next input character, we increase `i`.

```
i++;
```

29. Meaning of `j`

The variable `j` points to the next free position in the postfix array.

Example:

```
postfix: a b c * +
         ^
         j
```

When we write a character into postfix, we increase `j`.

```
j++;
```

30. Why We Need Both `i` and `j`

The input expression and output expression are not built at the same speed.

Sometimes we read one character and write one character.

Example:

```
Read operand a -> write a
```

But sometimes we may pop an operator from the stack without moving to the next input character.

Example:

```
current input = +  
stack top = *
```

We pop `*`, but we still have not handled `+`.

So `i` and `j` must be separate.

Important idea:

`i` tracks reading.

`j` tracks writing.

Reading and writing do not always happen together.

31. Creating the Stack Inside `infixToPostfix()`

The main conversion function begins by creating a stack.

```
Stack st;  
st.size = strlen(infix);  
st.top = -1;  
st.S = new char[st.size];
```

Explanation:

Step 1: Create the stack variable

```
Stack st;
```

This creates a stack named `st`.

Step 2: Set the stack size

```
st.size = strlen(infix);
```

`strlen(infix)` gives the number of visible characters in the infix expression.

Example:

```
infix = "a+b*c"
```

The visible characters are:

```
a + b * c
```

There are 5 characters.

So:

```
strlen(infix) = 5
```

Step 3: Set the stack to empty

```
st.top = -1;
```

This means the stack starts empty.

Step 4: Allocate the stack array

```
st.S = new char[st.size];
```

This creates a character array for the stack.

32. Why Stack Size Is `strlen(infix)`

The stack is used to store operators.

In the worst case, many characters could be pushed.

Using the length of the expression as the stack size is safe for this beginner version.

Example:

```
infix = "a+b*c"  
length = 5
```

So the stack has enough space for the operators in the expression.

33. Creating the Postfix Array

The postfix result is stored in another character array.

```
char *postfix = new char[strlen(infix) + 1];
```

Why `+ 1`?

Because C-style strings need a null terminator at the end:

```
'\0'
```

Example:

```
postfix = "abc*+"
```

This is stored as:

```
'a' 'b' 'c' '*' '+' '\0'
```

The visible expression has 5 characters.

But the array needs 6 spaces because of `\0`.

So we allocate:

```
strlen(infix) + 1
```

34. What Is `\0`?

`\0` is the **null terminator**.

It marks the end of a C-style string.

Example:

```
char text[] = "abc";
```

In memory, it is stored like this:

```
'a' 'b' 'c' '\0'
```

The `\0` tells the program:

```
The string ends here.
```

Without `\0`, the program may keep reading memory beyond the end of the string.

That can cause incorrect output.

35. Main Conversion Function

The main conversion function is:

```
char* infixToPostfix(char *infix)
```

It receives an infix expression and returns a postfix expression.

Full function:

```
char* infixToPostfix(char *infix) {
    Stack st;
    st.size = strlen(infix);
    st.top = -1;
    st.S = new char[st.size];

    char *postfix = new char[strlen(infix) + 1];

    int i = 0; // scans infix
    int j = 0; // writes postfix

    while (infix[i] != '\0') {
```

```

        if (isOperand(infix[i])) {
            postfix[j++] = infix[i++];
        } else {
            if (precedence(infix[i]) > precedence(stackTop(st))) {
                push(&st, infix[i++]);
            } else {
                postfix[j++] = pop(&st);
            }
        }
    }

    while (!isEmpty(st)) {
        postfix[j++] = pop(&st);
    }

    postfix[j] = '\0';

    delete[] st.S;

    return postfix;
}

```

Now we will explain this function slowly.

36. The Main `while` Loop

The program scans the infix expression using this loop:

```

while (infix[i] != '\0') {
    ...
}

```

Meaning:

Keep scanning until the end of the infix string.

The loop stops when it reaches:

`'\0'`

That means the expression has ended.

37. Case 1: Current Character Is an Operand

Inside the loop, first we check:

```
if (isOperand(infix[i]))
```

If this is true, the current character is an operand.

Example:

```
a  
b  
c
```

Operands go directly to the postfix output.

Code:

```
postfix[j++] = infix[i++];
```

This line means:

1. Copy `infix[i]` into `postfix[j]`.
2. Increase `j`.
3. Increase `i`.

Example:

```
infix[i] = 'a'
```

Then:

```
postfix[j] = 'a'
```

After that:

```
i moves to the next input character  
j moves to the next output position
```

38. Why Operand Goes Directly to Postfix

In postfix notation, operands appear in the same order they are needed.

Example:

a + b

Postfix:

ab+

The operands `a` and `b` appear before the operator.

So when we scan operands, we can immediately copy them to the postfix result.

No need to push operands into the stack in this beginner method.

Important rule:

Operand -> postfix directly

39. Case 2: Current Character Is an Operator

If the current character is not an operand, then in this beginner program it is treated as an operator.

Example operators:

+ - * /

Now the program compares the current operator with the operator on top of the stack.

Code:

```
if (precedence(infix[i]) > precedence(stackTop(st))) {  
    push(&st, infix[i++]);  
} else {
```

```
    postfix[j++] = pop(&st);  
}
```

There are two subcases.

40. Operator Case A: Current Operator Has Higher Precedence

If the current operator is stronger than the stack top, push it.

Condition:

```
precedence(infix[i]) > precedence(stackTop(st))
```

Example:

```
infix = a+b*c
```

After reading `a+b`, we have:

```
postfix = ab  
stack = +  
current input = *
```

Compare:

```
current operator = *  
stack top = +
```

Precedence:

```
* = 2  
+ = 1
```

Since:

```
2 > 1
```

we push `*`.

Now:

```
stack = + *  
postfix = ab
```

Important idea:

A stronger operator should wait above the weaker operator, because it must be output first later.

41. Operator Case B: Current Operator Has Lower or Equal Precedence

If the current operator is weaker or equal compared to the stack top, pop from the stack into postfix.

Example:

```
infix = a*b+c
```

After reading `a*b`, we have:

```
postfix = ab  
stack = *  
current input = +
```

Compare:

```
current operator = +  
stack top = *
```

Precedence:

```
+ = 1  
* = 2
```

Since `+` is weaker than `*`, we pop `*` first.

Now:

```
postfix = ab*  
stack = empty
```

But `+` is still not handled yet.

So the program does not increase `i` when it pops.

The next loop checks the same `+` again.

42. Why Equal Precedence Causes Pop

In this beginner version, the operators `+`, `-`, `*`, and `/` are treated as left-to-right operators.

Example:

```
a-b-c
```

This means:

```
(a-b)-c
```

not:

```
a-(b-c)
```

Correct postfix:

```
ab-c-
```

When the program reaches the second `-`, the first `-` is already in the stack.

Both have equal precedence.

Because subtraction is left-to-right, the first `-` must be output before the second `-`.

So equal precedence should cause a pop.

In Lesson 16 code, this behavior happens because the program only pushes when the current operator has strictly higher precedence:

```
precedence(infix[i]) > precedence(stackTop(st))
```

If precedence is equal, the condition is false.

So the program pops.

43. Why the Code Uses `>` Instead of `>=`

The program uses:

```
precedence(infix[i]) > precedence(stackTop(st))
```

not:

```
precedence(infix[i]) >= precedence(stackTop(st))
```

Why?

Because if the current operator has equal precedence, we should pop the previous operator first for left-to-right operators.

Example:

```
a-b-c
```

If we used `>=`, the second `-` would be pushed above the first `-`.

That could give the wrong order.

So for this beginner version:

```
Higher precedence -> push  
Lower or equal precedence -> pop
```

44. Why Lesson 16 Uses a `while` Loop, Not a `for` Loop

This is one of the most important ideas in Lesson 16.

A normal `for` loop usually increases `i` automatically after every loop cycle.

Example:

```
for (int i = 0; infix[i] != '\0'; i++)
```

But in infix-to-postfix conversion, sometimes we must not move to the next input character yet.

Example:

```
a*b+c
```

When we reach `+`, the stack contains `*`.

```
current input = +  
stack top     = *
```

Since `*` is stronger, we pop `*` into postfix.

But we have not handled the `+` yet.

So `i` must stay on `+`.

Then the program compares `+` again with the new stack top.

That is why we use:

```
while (infix[i] != '\0')
```

instead of a `for` loop.

45. The Rule for Moving `i`

The simple rule is:

```
If you copy an operand, increment i.  
If you push the current operator, increment i.  
If you pop from the stack, do not increment i.
```

Why?

Because:

```
Copying an operand consumes the current input character.  
Pushing an operator consumes the current input character.  
Popping from the stack does not consume the current input character.
```

This is why manual control of `i` is needed.

46. Understanding `postfix[j++] = infix[i++]`

This line may look confusing:

```
postfix[j++] = infix[i++];
```

It means:

```
postfix[j] = infix[i];  
j = j + 1;  
i = i + 1;
```

Example:

Before:

```
i = 0  
j = 0  
infix[i] = 'a'
```

After:

```
postfix[j++] = infix[i++];
```

we get:

```
postfix[0] = 'a'  
i = 1  
j = 1
```

47. Understanding `push(&st, infix[i++])`

This line means:

```
push(&st, infix[i++]);
```

It does two things:

1. Push the current input character into the stack.
2. Move `i` to the next input character.

Example:

Before:

```
infix[i] = '+'
```

After:

```
push(&st, infix[i++]);
```

we get:

```
'+' is pushed into stack  
i moves to the next character
```

48. Understanding `postfix[j++] = pop(&st)`

This line means:

```
postfix[j++] = pop(&st);
```

It does two things:

1. Pop the top operator from the stack.
2. Store it in postfix[j].
3. Move j to the next output position.

Important:

```
i does not change here.
```

Why?

Because we did not consume the current input character.

We only removed an operator from the stack.

49. After the Main Scan Ends

After the main loop finishes, the input expression has been fully scanned.

But the stack may still contain operators.

Example:

```
infix = a+b*c
```

After scanning all characters, the stack contains:

```
+ *
```

These operators must be moved to postfix.

So we use:

```
while (!isEmpty(st)) {  
    postfix[j++] = pop(&st);  
}
```

This means:

While the stack is not empty, pop operators into postfix.

50. Why Remaining Operators Are Popped at the End

Operators in the stack are waiting.

At the end of the expression, there are no more input characters.

So every remaining operator must now be output.

Example:

```
postfix = abc  
stack = + *
```

Pop *****:

```
postfix = abc*  
stack = +
```

Pop **+**:

```
postfix = abc*+  
stack = empty
```

Final postfix:

```
abc*+
```

51. Ending the Postfix String

After all characters are written into postfix, we must add:

```
postfix[j] = '\0';
```

This marks the end of the postfix string.

Without this line, printing the postfix expression may produce extra garbage characters.

Example:

```
postfix array: 'a' 'b' 'c' '*' '+' '\0'
```

The `\0` says:

```
Stop reading here.
```

52. Cleaning Up Stack Memory

The stack array was created using:

```
st.S = new char[st.size];
```

So after we finish using it, we should release it:

```
delete[] st.S;
```

This prevents memory leaks.

Important rule:

```
If you allocate with new[], release with delete[].
```

53. Why We Do Not Delete `postfix` Inside the Function

The function returns `postfix`:

```
return postfix;
```

So we cannot delete it inside `infixToPostfix()` before returning.

If we deleted it inside the function, the caller would receive an invalid pointer.

Instead, the caller deletes it later.

Example in `main()`:

```
char *postfix = infixToPostfix(infix);  
cout << postfix;  
delete[] postfix;
```

54. Complete Beginner C++-Style Program

```
#include <iostream>  
#include <cstring>  
using namespace std;  
  
struct Stack {  
    int size;  
    int top;  
    char *S;  
};  
  
int isEmpty(Stack st) {  
    return st.top == -1;  
}  
  
int isFull(Stack st) {  
    return st.top == st.size - 1;  
}  
  
void push(Stack *st, char x) {  
    if (isFull(*st)) {  
        cout << "Stack Overflow\n";  
    } else {
```

```

        st->top++;
        st->S[st->top] = x;
    }
}

char pop(Stack *st) {
    char x = '\0';

    if (isEmpty(*st)) {
        cout << "Stack Underflow\n";
    } else {
        x = st->S[st->top];
        st->top--;
    }

    return x;
}

char stackTop(Stack st) {
    if (isEmpty(st))
        return '\0';

    return st.S[st.top];
}

int isOperand(char x) {
    if (x == '+' || x == '-' || x == '*' || x == '/')
        return 0;
    return 1;
}

int precedence(char x) {
    if (x == '+' || x == '-')
        return 1;
    else if (x == '*' || x == '/')
        return 2;
    return 0;
}

char* infixToPostfix(char *infix) {
    Stack st;
    st.size = strlen(infix);
    st.top = -1;
    st.S = new char[st.size];

    char *postfix = new char[strlen(infix) + 1];

    int i = 0; // scans infix

```



```

int j = 0; // writes postfix

while (infix[i] != '\0') {
    if (isOperand(infix[i])) {
        postfix[j++] = infix[i++];
    } else {
        if (precedence(infix[i]) > precedence(stackTop(st))) {
            push(&st, infix[i++]);
        } else {
            postfix[j++] = pop(&st);
        }
    }
}

while (!isEmpty(st)) {
    postfix[j++] = pop(&st);
}

postfix[j] = '\0';

delete[] st.S;

return postfix;
}

int main() {
    char infix[] = "a+b*c";

    char *postfix = infixToPostfix(infix);

    cout << "Infix: " << infix << endl;
    cout << "Postfix: " << postfix << endl;

    delete[] postfix;

    return 0;
}

```

55. Expected Output

For this input:

```
char infix[] = "a+b*c";
```

The output is:

Infix: a+b*c
Postfix: abc*+

56. Dry Run 1: a+b*c

Expression:

a+b*c

Expected postfix:

abc*+

Initial state:

i = 0
j = 0
stack = empty
postfix = empty

Trace:

Step	Current character	Action	Stack after action	Postfix after action
1	a	Operand, copy to postfix	empty	a
2	+	Stack empty, push +	+	a
3	b	Operand, copy to postfix	+	ab
4	*	* has higher precedence than +, push *	+ *	ab
5	c	Operand, copy to postfix	+ *	abc
End	—	Pop *	+	abc*
End	—	Pop +	empty	abc*+

Final postfix:

abc*+

So:

a+b*c -> abc*+

Why?

Because multiplication happens before addition:

a + (b * c)

57. Dry Run 2: a*b+c

Expression:

a*b+c

Expected postfix:

ab*c+

Initial state:

stack = empty
postfix = empty

Trace:

Step	Current character	Action	Stack after action	Postfix after action
1	a	Operand, copy to postfix	empty	a
2	*	Stack empty, push *	*	a
3	b	Operand, copy to postfix	*	ab
4	+	+ is weaker than *, pop *	empty	ab*

Step	Current character	Action	Stack after action	Postfix after action
5	+	Now stack empty, push +	+	ab*
6	c	Operand, copy to postfix	+	ab*c
End	—	Pop +	empty	ab*c+

Final postfix:

ab*c+

Important moment:

At step 4, + was not consumed yet.

The program popped *, but i stayed on +.

Then the program checked + again and pushed it.

This is the reason we use a while loop.

58. Dry Run 3: a+b-c

Expression:

a+b-c

Expected postfix:

ab+c-

Why?

+ and - have the same precedence.

They are handled left to right.

So:

a+b-c

means:

(a+b)-c

Trace:

Step	Current character	Action	Stack after action	Postfix after action
1	a	Operand, copy	empty	a
2	+	Push +	+	a
3	b	Operand, copy	+	ab
4	-	Equal precedence with +, pop +	empty	ab+
5	-	Stack empty, push -	-	ab+
6	c	Operand, copy	-	ab+c
End	—	Pop -	empty	ab+c-

Final postfix:

ab+c-

59. Dry Run 4: a+b*c-d/e

Expression:

a+b*c-d/e

Expected postfix:

abc*+de/-

Reason:

a + (b*c) - (d/e)

Trace:

Step	Current character	Action	Stack after action	Postfix after action
1	a	Operand, copy	empty	a
2	+	Push +	+	a
3	b	Operand, copy	+	ab
4	*	* stronger than +, push *	+ *	ab
5	c	Operand, copy	+ *	abc
6	-	- weaker than *, pop *	+	abc*
7	-	- equal to +, pop +	empty	abc*+
8	-	Stack empty, push -	-	abc*+
9	d	Operand, copy	-	abc*+d
10	/	/ stronger than -, push /	- /	abc*+d
11	e	Operand, copy	- /	abc*+de
End	—	Pop /	-	abc*+de/
End	—	Pop -	empty	abc*+de/-

Final postfix:

abc*+de/-

60. Why the Stack Is Needed in a+b*c-d/e

Look at this part:

a+b*c

The + is seen before *, but * must happen first.

So + waits in the stack.

Then `*` is pushed above it.

At the right time, `*` is popped before `+`.

This is exactly why a stack is useful.

61. What This Beginner Program Handles

This Lesson 16 beginner version handles:

```
single-character operands
operators + - * /
operator precedence
left-to-right behavior for equal precedence
basic infix-to-postfix conversion
```

It works for expressions like:

```
a+b*c
a*b+c
a+b-c
a+b*c-d/e
x/y+z
```

62. What This Beginner Program Does Not Handle Yet

This beginner version does not yet handle:

```
parentheses
spaces
multi-digit numbers
unary operators
power operator
function names
invalid expressions
```

Examples it does not properly handle yet:

```
(a+b)*c
12+34*5
-a+b
a^b^c
logN+M
```

These require extra rules.

63. Why Parentheses Are Not Handled Yet

This Lesson 16 version only compares operator precedence.

Parentheses require special rules:

```
If '(' appears, push it.
If ')' appears, pop until '(' is found.
```

Those rules are not included in the simple Lesson 16 program.

So this expression is not supported yet:

```
(a+b)*c
```

64. Why Multi-Digit Numbers Are Not Handled Yet

This program scans one character at a time.

Example:

```
12+34
```

The program sees:

```
'1', '2', '+', '3', '4'
```

It does not understand that `12` is one number and `34` is another number.

To support multi-digit numbers, the program would need tokenization.

Tokenization means grouping characters into meaningful units.

Example:

12+34

Tokens:

12, +, 34

That is beyond this beginner version.

65. Why Unary Operators Are Not Handled Yet

Unary operators work on one operand.

Example:

-a

Here, means negative sign, not subtraction.

But the Lesson 16 program treats every as binary subtraction.

So it does not properly handle:

-a+b

Unary operators require extra logic to decide whether means:

negative sign

or:

subtraction

66. Time Complexity

The time complexity is:

$O(n)$

where n is the length of the infix expression.

Why?

Because:

Each input character is scanned.
Each operator is pushed at most once.
Each operator is popped at most once.

So the work grows linearly with the input size.

Example:

If the expression has 10 characters, the work is proportional to 10.
If the expression has 100 characters, the work is proportional to 100.

So the time complexity is:

$O(n)$

67. Space Complexity

The space complexity is:

$O(n)$

Why?

Because the program creates:

1. A stack array
2. A postfix output array

Both depend on the length of the input expression.

So if the input has `n` characters, the extra memory can also grow with `n`.

Therefore:

Space complexity = $O(n)$

68. Common Beginner Mistakes

Mistake 1: Moving `i` after every loop

Wrong idea:

Always increase `i` at the end of the loop.

This is wrong because sometimes we pop from the stack but still need to process the same input operator.

Correct idea:

Increase `i` only when the current input character is consumed.

Mistake 2: Forgetting to pop remaining operators

If you forget this part:

```
while (!isEmpty(st)) {  
    postfix[j++] = pop(&st);  
}
```

then some operators may remain in the stack.

Example:

```
a+b*c
```

You might get:

```
abc
```

instead of:

```
abc*+
```

Mistake 3: Forgetting the null terminator

If you forget:

```
postfix[j] = '\0';
```

then printing postfix may show extra garbage characters.

Correct idea:

```
Always end a C-style string with \0.
```

Mistake 4: Using `>=` instead of `>` in the push condition

This beginner version uses:

```
precedence(infix[i]) > precedence(stackTop(st))
```

If you use `>=`, equal-precedence left-to-right operators may be handled incorrectly.

Example:

```
a-b-c
```

Correct grouping:

```
(a-b)-c
```

Correct postfix:

```
ab-c-
```

Mistake 5: Not deleting dynamic memory

If you allocate memory using:

```
new char[...]
```

then you should release it using:

```
delete[] ...
```

Otherwise, the program may leak memory.

Mistake 6: Thinking the stack stores the final answer

The stack does not store the final answer.

The final answer is stored in:

```
postfix
```

The stack only stores operators temporarily.

Mistake 7: Thinking operands are pushed into the stack

In this beginner method, operands are not pushed into the stack.

Operands go directly to postfix.

Only operators are pushed into the stack.

69. Key Rules to Memorize

Operand -> copy directly to postfix.
Operator -> compare with stack top.
Higher precedence -> push.
Lower or equal precedence -> pop.
At the end -> pop all remaining operators.

For index movement:

Copy operand -> move i and j.
Push operator -> move i.
Pop operator -> move j only.

For strings:

End postfix with \0.

For memory:

new[] must be matched with delete[].

70. Simple Mental Model

Think of the program like this:

The infix expression is the input line.
The postfix array is the answer sheet.
The stack is a waiting room for operators.

When an operand appears:

Send it directly to the answer sheet.

When an operator appears:

Check whether it should wait in the stack or whether older operators must leave first.

When the input ends:

Empty the waiting room into the answer sheet.

71. Final Summary

Lesson 16 teaches how to program infix-to-postfix conversion using character arrays and a stack.

The main idea is:

Operands go directly to postfix.
Operators are temporarily stored in the stack.
Precedence decides when operators are pushed or popped.

The variables `i` and `j` are very important:

`i` scans the infix expression.
`j` writes into the postfix expression.

The program uses a `while` loop because `i` does not always move forward.

Sometimes the program pops from the stack while staying on the same input operator.

The final postfix expression must end with:

`'\0'`

The stack memory must be released with:

`delete[] st.S;`

The returned postfix memory must be released later by the caller.

Time complexity:

$O(n)$

Space complexity:

$O(n)$

72. Beginner Takeaway

The most important Lesson 16 takeaway is:

```
i reads the infix expression.  
j writes the postfix expression.  
Operands are copied immediately.  
Operators wait in the stack.  
When popping happens, i does not move.
```

Once you understand this, the code becomes much easier to follow.